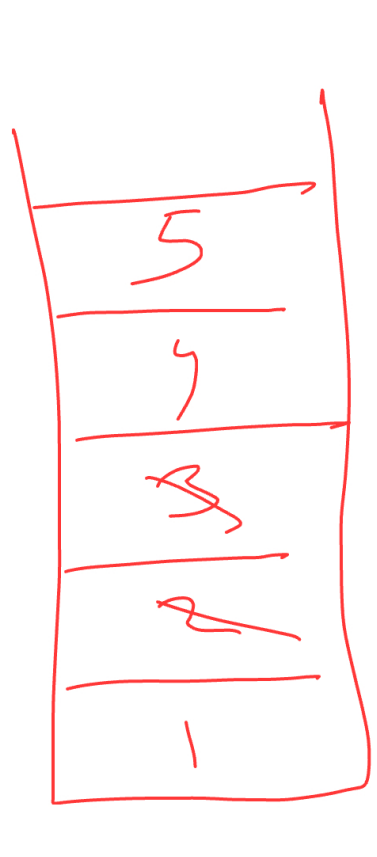
LIFO → Last In First Out

Push(1)
Push(2)
Push(3)
pop() → 3
pop() → 2
push(4)
push(5)
top() → 5



- Iterations over stack are not possible
- If you want to access the bottommost element you will have to delete everything else to reach the bottom.

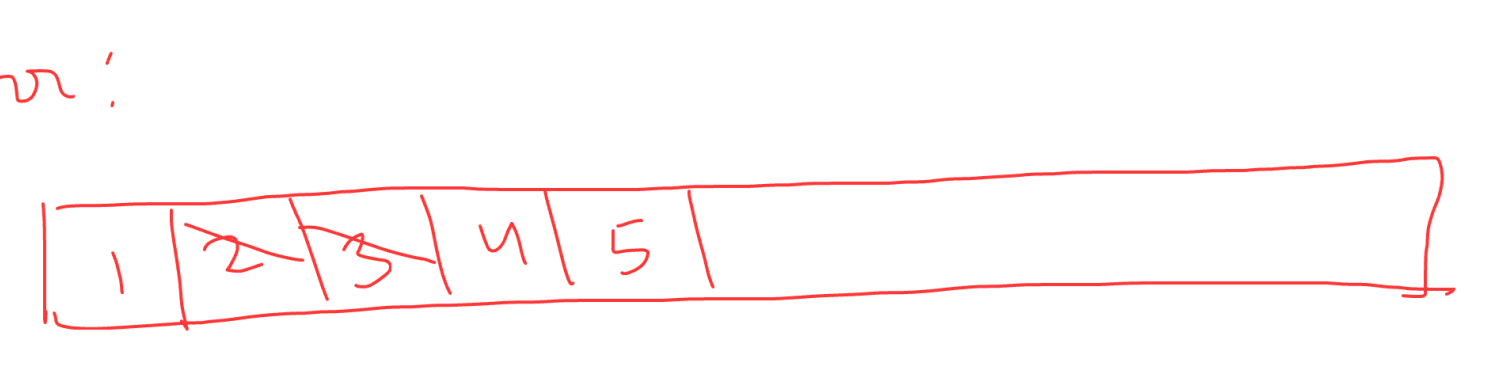
---

## Stacks using Arrays

```
class Stack {
    private:
        vector <int> arr;

    public:
        push()
        pop()
        top()
        size()
        isEmpty()
};
```

Push(1)
Push(2)
Push(3)
pop() → 3
pop() → 2
push(4)
push(5)
top() → 5

arr:



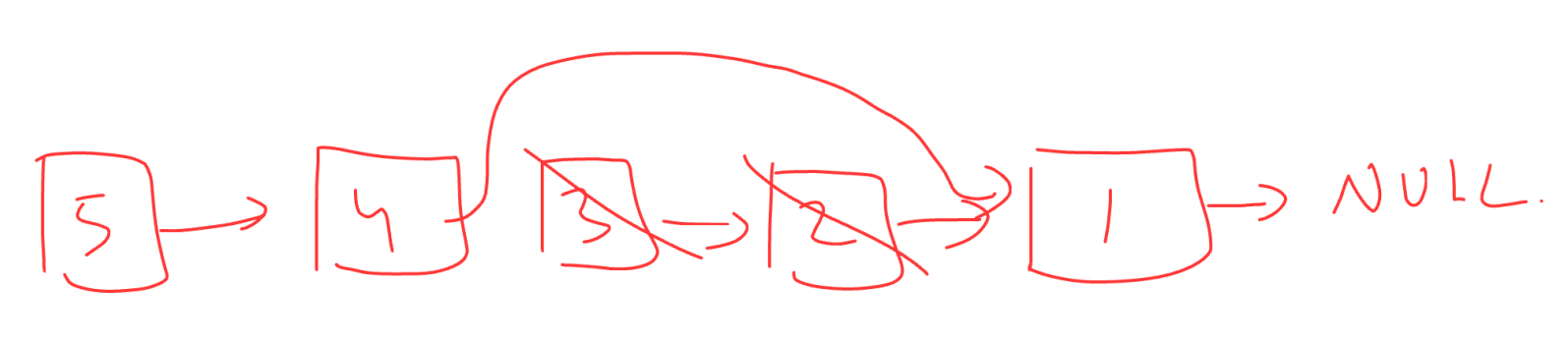
| Stack | Array |
|---|---|
| O(1) ← push() | push-back() |
| O(1) ← pop() | pop-back() |
| O(1) ← top() | return arr[arr.size()-1] |

---

## Stacks Using L.L.

```
class Stack {
    private:
        LinkedList ll;
    public:
        push()
        pop()
        top()
        :
};
```

Push(1)
Push(2)
Push(3)
pop() → 3
pop() → 2
push(4)
push(5)
top() → 5



| Stack | L.L. |
|---|---|
| O(1) ← push() | push at Begin() |
| O(1) ← pop() | delete at Begin() |
| O(1) ← top() | return head → data. |

---

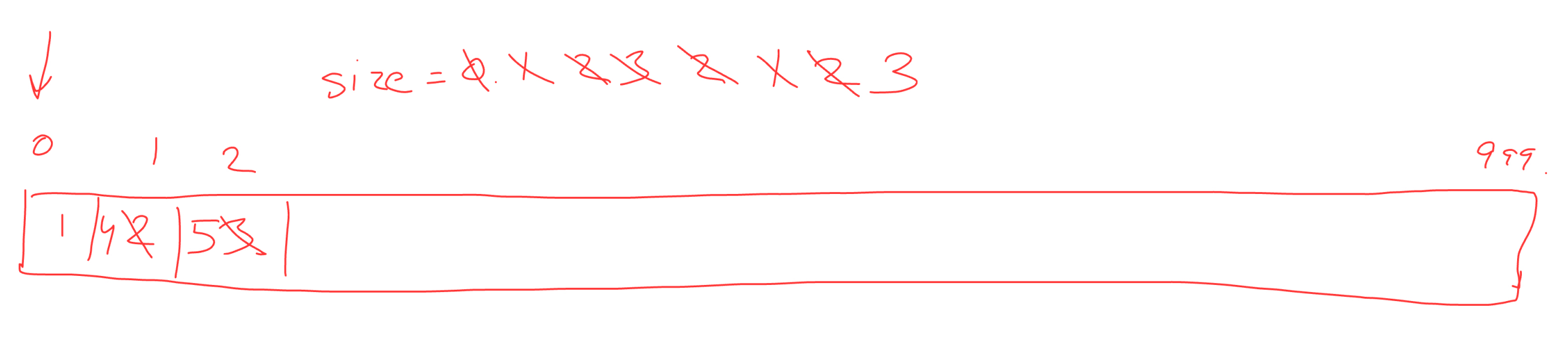
## Stack Using Static Arrays.

suppose max size = 1000.

```
class Stack {
    private:
        int arr[1000];
        int size;
    public:
        :
}
```

Push(1)
Push(2)
Push(3)
pop() → 3
pop() → 2
push(4)
push(5)
top() → 5

size = ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~2~~ ~~1~~ ~~2~~ 3



| Stack | Static Array |
|---|---|
| O(1) ← push() | arr[size-1] = value. size++ |
| O(1) ← pop() | size-- |
| O(1) ← top() | arr[size-1]. |

---

( )   { }   [ ]

( ) { } → True.

{ ( ) } → True

{ ( ) ] → False.

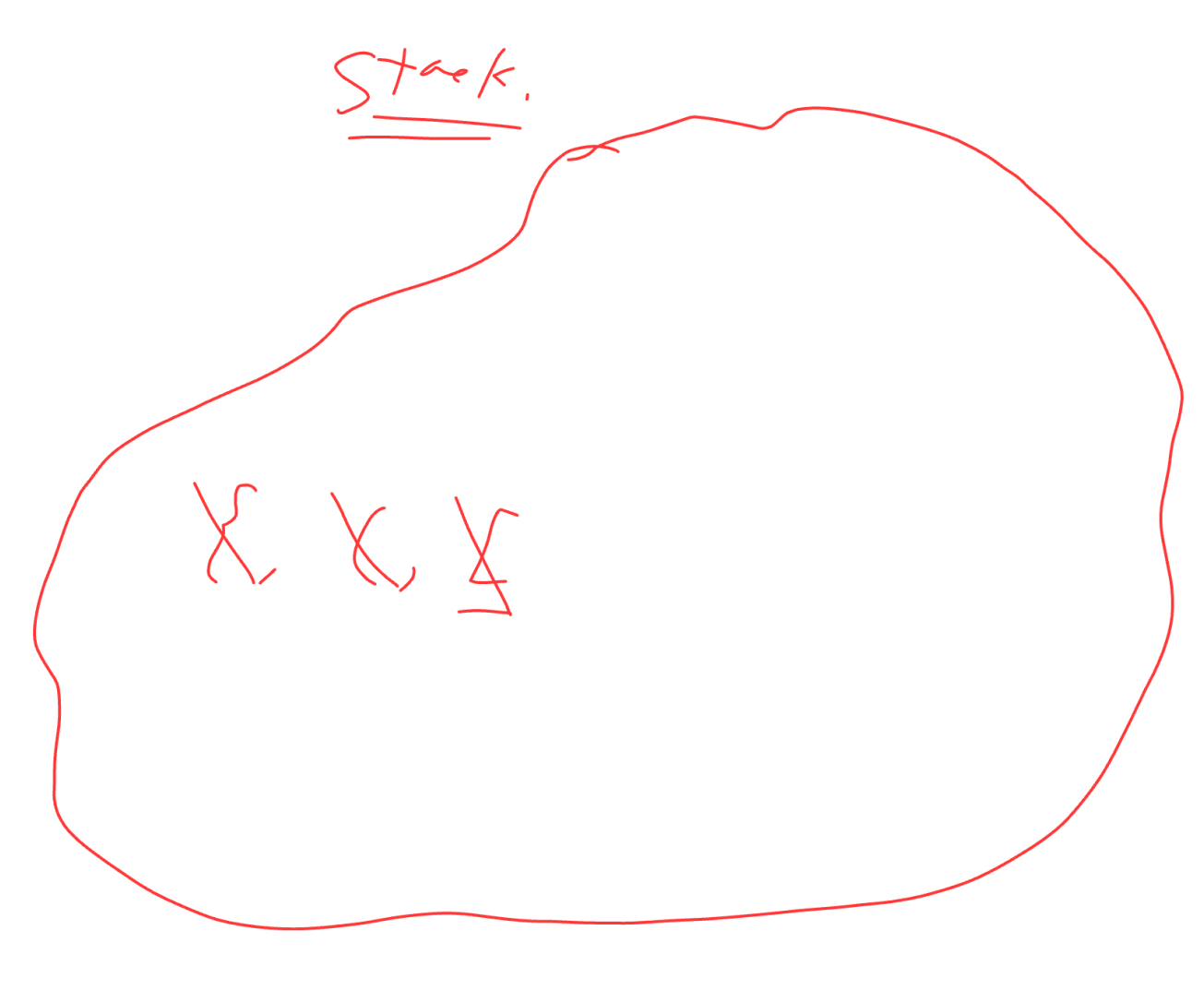
{ } ( → False.

{ ( [ ] ) }

( ) [ ]

{ ( [ ] ) }

Stack.



[ { } ( [ ] ) ]

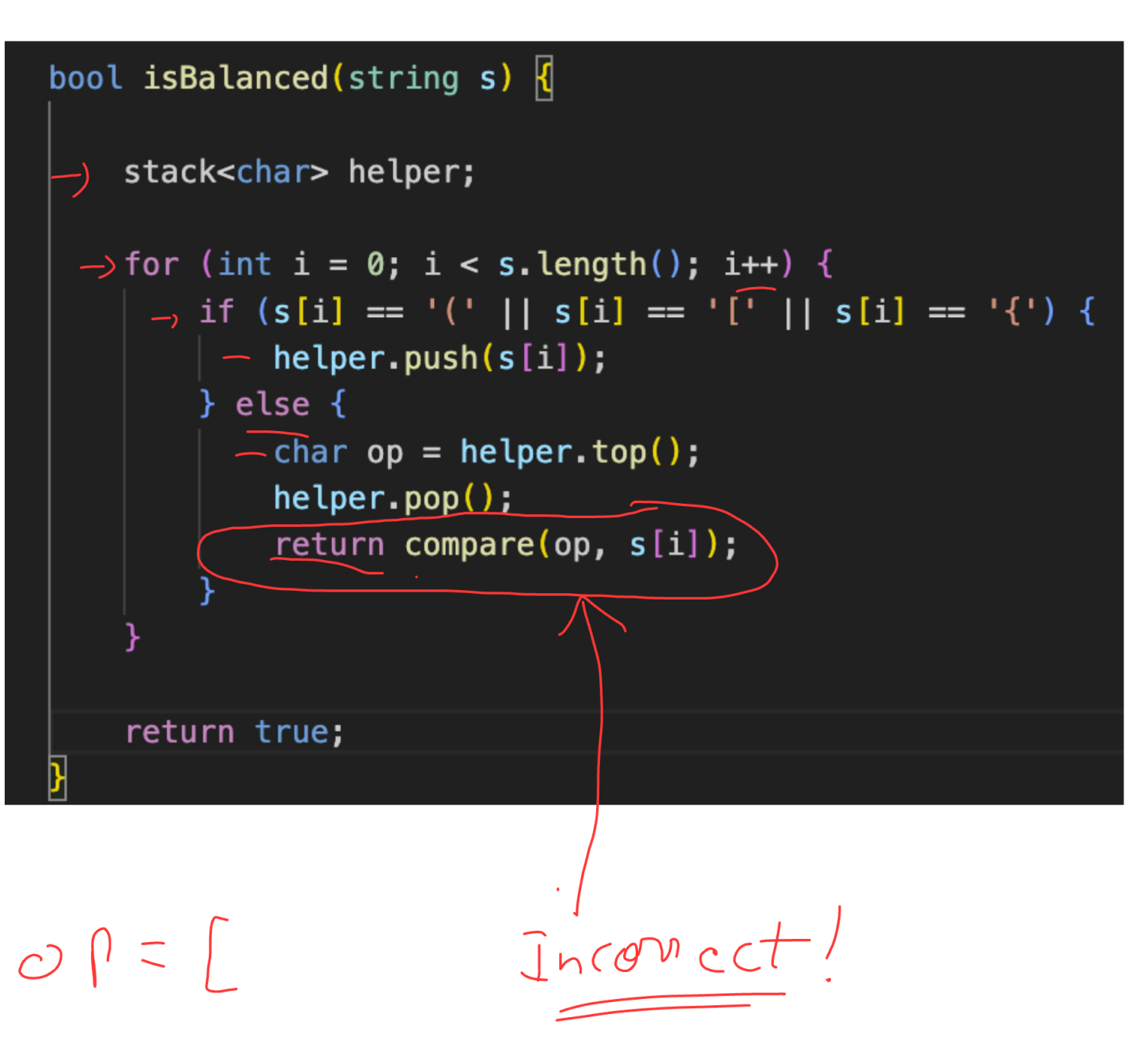
Stack.

TC = O(n)
AS = O(n).

"((((((([{([("

(op, cl)

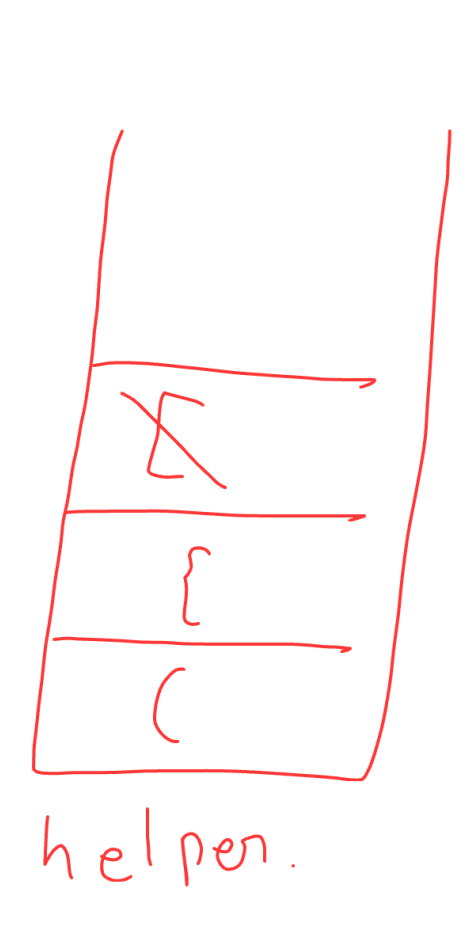
return (op == '[' && cl == ']') ||
(op == '{' && cl == '}') ||
(op == '(' && cl == ')');



```
bool isBalanced(string s) {

    stack<char> helper;

    for (int i = 0; i < s.length(); i++) {
        if (s[i] == '(' || s[i] == '[' || s[i] == '{') {
            helper.push(s[i]);
        } else {
            char op = helper.top();
            helper.pop();
            return compare(op, s[i]);
        }
    }

    return true;
}
```

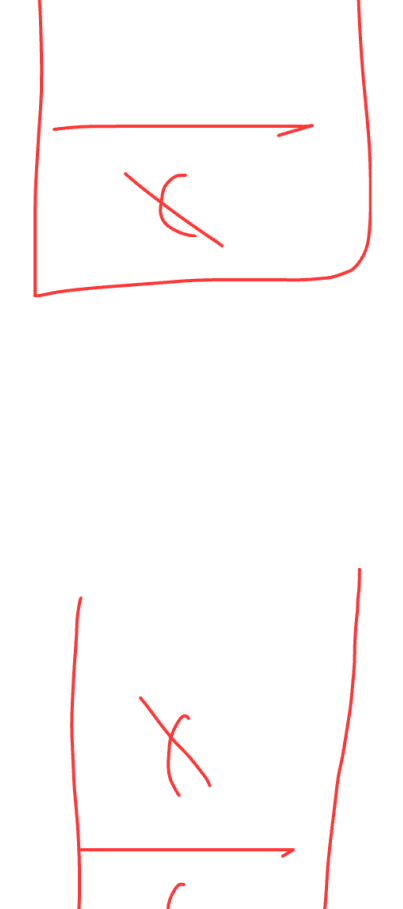
Incorrect!

( { [ ] ) }

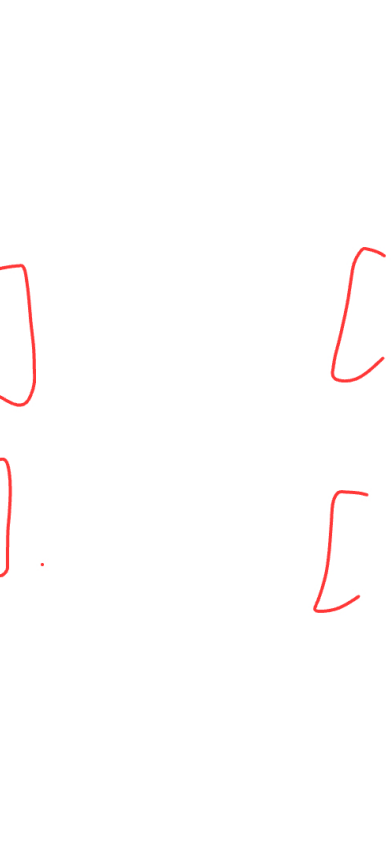


op = [

( ) ]



( ( ( )



---

## N.G.E.

I/P: [ 6, 8, 0, 1, 3 ]
O/P: [ 8, -1, 1, 3, -1 ]

[ 6, 8, 0, 3, 1 ]
[ 8, -1, 3, -1, -1 ]

## B.F

```
i: 0 ⟶ n-1
    j: i+1 ⟶ n-1
        if (arr[j] > arr[i]) {
            nge[i] = arr[j]
            break;
```

TC = O(n²)
AS = O(1)

I/P: [ 6, 8, 0, 1, 3, 9 ]
O/P: [ 8, 9, 1, 3, 9, -1 ]

[ 6, 8, 0, 1, 3, 9 ]
8 9 1 3 9 -1

Stack.